

CSC1031

Computer Programming**4**

Object Oriented Programming

Mikhail Kudriavtsev
Misha

Email: Mikhail.kudriavtsev@dcu.ie

How Compiling Works with Multiple Classes in One File

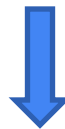
- **File Structure with Multiple Classes:**

- Only one class can be **public** and must match the filename.
- Other classes are package-private and accessible only within the same package.

- **Compilation Process:**

- Compiling the file generates separate **.class** files for each class.

MainClass.java



```
.  
├── MainClass.class  
└── HelperClass.class
```

```
public class MainClass {  
    public static void main(String[] args) {  
        HelperClass.sayHello();  
    }  
}  
  
class HelperClass {  
    public static void sayHello() {  
        System.out.println("Hello from HelperClass!");  
    }  
}
```

What Does `Fridge fridge = new Fridge(50);` Mean?

class name
Your data type

variable name
reference
Holds the memory
address of the
Fridge object.

keyword
used to create
a new object in
the memory

call to the **constructor** of the
Fridge class.
Initializes the Fridge object
with a balance of 50 (or other
parameters).

`int a = 5;`

Initial value

`Fridge b = new Fridge(50);`

`Fridge abc = new Fridge(50);`

`Fridge fRiDgE = new Fridge(50);`



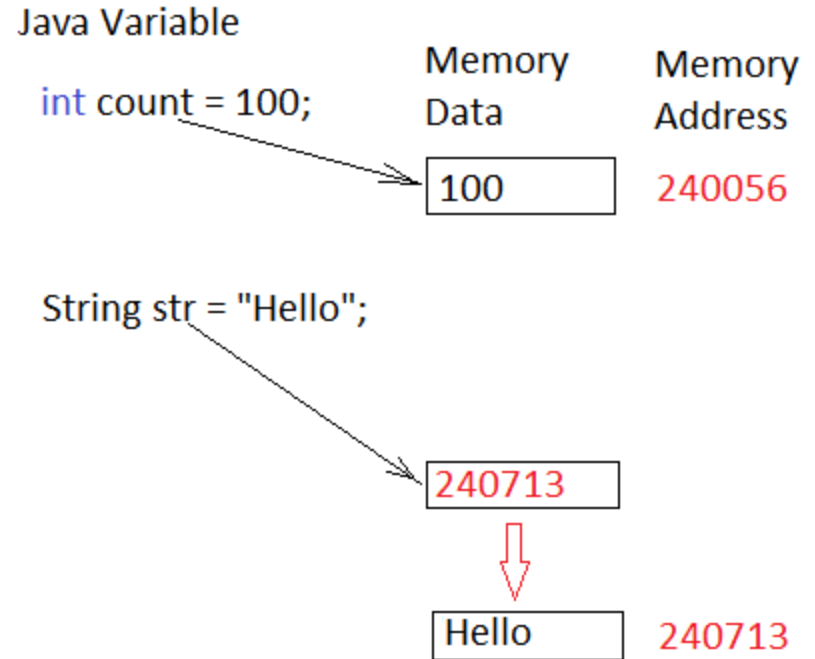
Reference Types in Java

What is a Reference Type?

- A variable that stores the **memory address** of an object, not the object itself.
- Examples: Strings, Arrays, Custom Objects.

Passing References to Methods:

- Java passes **a copy of the reference** (not the actual object).
- Changes made to the object **inside the method** affect the original object.



Passing primitive types to methods

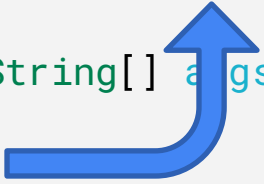
```
public class PrimitiveExample{  
    public static void modifyValue(int num) {  
        num = 100; // This only changes the local copy inside the method  
    }  
  
    public static void main(String[] args) {  
        int number = 1;  
        modifyvalue(number);  
        System.out.println(number); // Output: 1 (unchanged)  
    }  
}
```

Passing primitive types to methods

- When *modifyValue(number)* is called, a **copy** of the value stored in *number* is passed to the method.
- The parameter *num* holds the copy of the value.
- Changing *num* inside the method does **not** affect the original *number* in the main method.
- **Primitive types are passed by value**, meaning changes inside the method are limited to the local copy.

```
public class PrimitiveExample{  
    public static void modifyValue(int num) {  
        num = 100; // This only changes the local copy inside the method  
    }  
}
```

```
public static void main(String[] args) {  
    int number = 1;  
    modifyValue(number);  
    System.out.println(number); // Output: 1 (unchanged)  
}
```



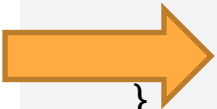
We sending a copy

Passing primitive types to methods

- When *modifyValue(number)* is called, a **copy** of the value stored in *number* is passed to the method.
- The parameter *num* holds the copy of the value.
- Changing *num* inside the method does **not** affect the original *number* in the main method.
- **Primitive types are passed by value**, meaning changes inside the method are limited to the local copy.

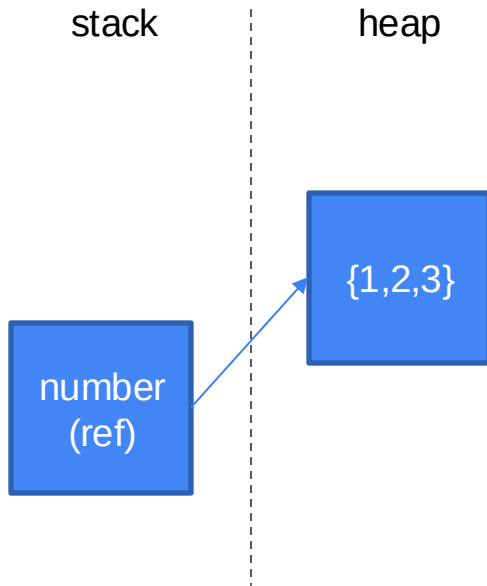
```
public class PrimitiveExample{
    public static void modifyValue(int num) {
        num = 100; // This only changes the local copy inside the method
    }

    public static void main(String[] args) {
        int number = 1;
        modifyValue(number);
        System.out.println(number); // Output: 1 (unchanged)
    }
}
```



Passing References to Methods

```
public class ReferenceExample {  
    public static void modifyArray(int[] arr) {  
        arr[0] = 100; // Changes the original array  
    }  
  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
        modifyArray(numbers);  
        System.out.println(numbers[0]); // Output: 100  
    }  
}
```



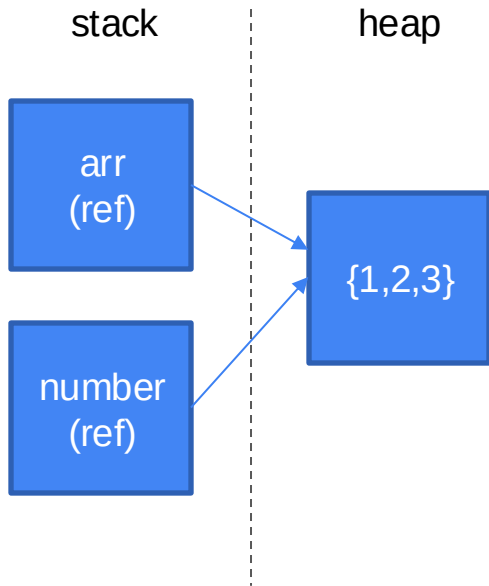
Passing References to Methods

`numbers` variable pointing to an array in memory

A method modifying the same array through its reference

```
public class ReferenceExample {  
    public static void modifyArray(int[] arr) {  
        arr[0] = 100; // Changes the original array  
    }  
  
    public static void main(String[] args)  
    {  
        int[] numbers = {1, 2, 3};  
        modifyArray(numbers);  
        System.out.println(numbers[0]); // Output: 100  
    }  
}
```

Copy of the reference

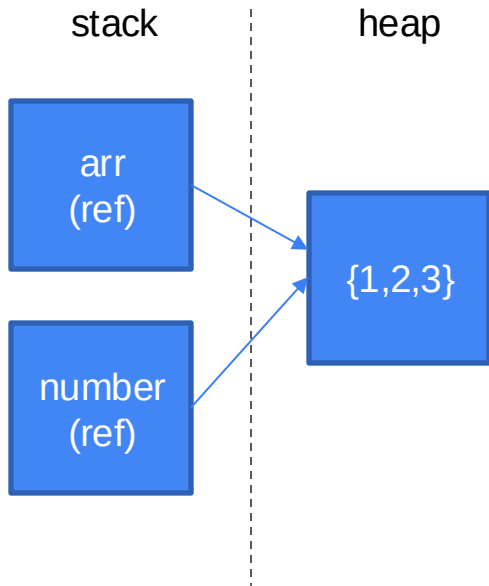
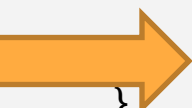


Passing References to Methods

`numbers` variable pointing to an array in memory

A method modifying the same array through its reference

```
public class ReferenceExample {  
    public static void modifyArray(int[] arr) {  
        arr[0] = 100; // Changes the original array  
    }  
  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
        modifyArray(numbers);  
        System.out.println(numbers[0]); // Output: 100  
    }  
}
```



Arrays and ArrayLists

Arrays:

- Fixed-size, reference-type behavior.
- Modifications within methods affect the original array.

ArrayLists:

- A resizable version of arrays, also reference types.
- Adding, removing, or modifying elements affects the original list.

Arrays and ArrayLists

We send REFERENCE

```
ArrayList<String> names = new ArrayList<>();  
modifyList(names);
```

```
int[] numbers = {1, 2, 3};  
modifyArray(numbers);
```

```
public static void modifyList(ArrayList<String> list)  
{  
    list.add("Hello");  
}
```

modifying elements affects the original

```
public static void modifyArray(int[] arr)  
{  
    arr[0] = 42;  
}
```

Shallow copy

- A copy of the **reference**, not the actual object.
- Both the original and the copied reference point to the **same object** in memory.
- Modifications to the object via one reference affect the other reference because they share the same memory.

```
Car car1 = new Car("Toyota");  
Car car2 = car1; // Both refer to the same object
```

Shallow copy

```
class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
}
```

Shallow copy

```
class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
    }  
}
```

Shallow copy

```
class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person original = new Person("Alice");  
        Person shallowPerson = original; // Shallow copy  
  
    }  
}
```


Shallow copy

both `original.name` and `shallowPerson.name` reflect the same value.

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Person original = new Person("Alice");
        Person shallowPerson = original; // Shallow copy

        shallowPerson.name = "Bob"; // Modifies the shared object
    }
}
```

Shallow copy

both `original.name` and `shallowPerson.name` reflect the same change.

```
class Person {
    String name;

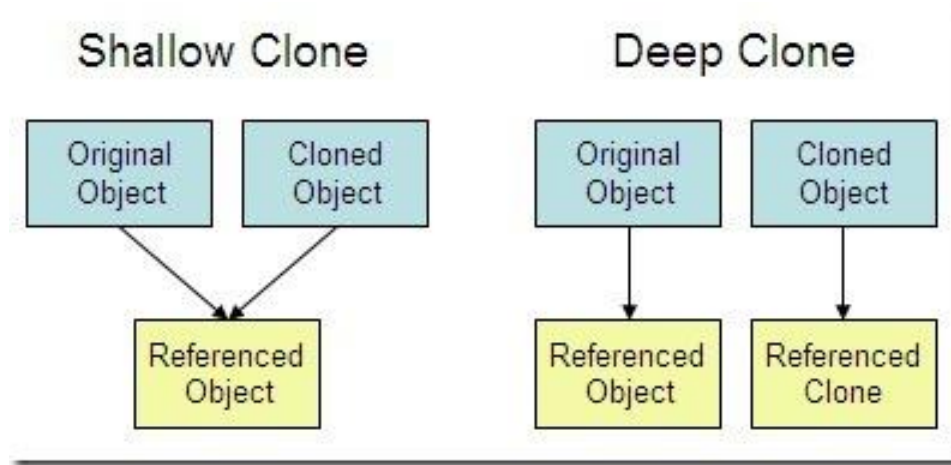
    Person(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Person original = new Person("Alice");
        Person shallowPerson = original; // Shallow copy

        shallowPerson.name = "Bob"; // Modifies the shared object
        System.out.println(original.name); // Output: Bob
    }
}
```

Deep copy

- A new object is created, and all fields are **manually copied** or copied using a constructor.
- The original and the copied objects are **completely independent** (no shared memory).
- Changes to one object do not affect the other.



```
class Car {
    String brand;
    double price;

    // Constructor
    public Car(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    // Method to create a deep copy of the object
    public Car copyCar() {

        return a; // Return the new independent object
    }
}

public static void main(String[] args) {
    // Original car
    Car original = new Car("Toyota", 30000.0);

    // Create a deep copy using the copyCar method
    Car copied = original.copyCar();
}
```

```
class Car {
    String brand;
    double price;

    // Constructor
    public Car(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    public Car(Car other) {
        this.brand = other.brand;
    }

    // Method to create a deep copy of the object
    public Car copyCar() {
        Car a = new Car(this.brand, this.price); // Use constructor to copy fields
        return a; // Return the new independent object
    }
}
```

```
class Car {  
    String brand;  
    double price;  
  
    // Constructor  
    public Car(String brand, double price) {  
        this.brand = brand;  
        this.price = price;  
    }  
}
```

```
public Car(Car other) {  
    this.brand = other.brand;  
}
```

Overloaded constructor
static polymorphism

```
    // Method to create a deep copy of the object  
    public Car copyCar() {  
        Car a = new Car(this.brand, this.price); // Use constructor to copy fields  
        return a; // Return the new independent object  
    }  
}
```

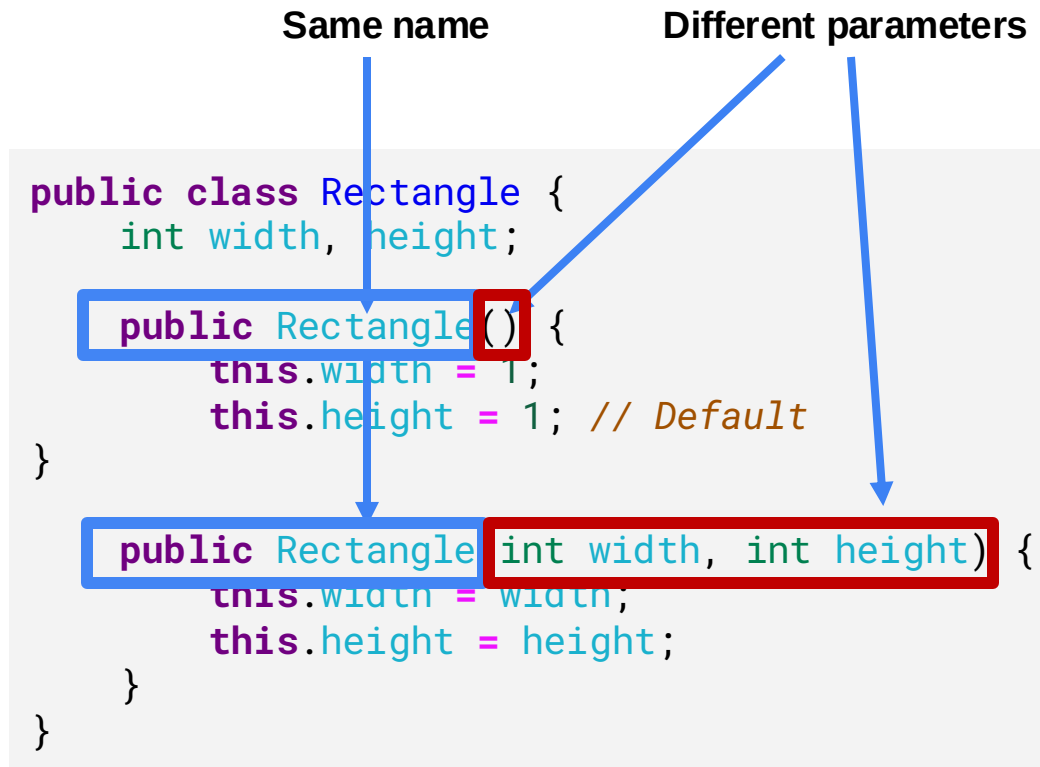
Overloading Constructors

What is Constructor Overloading?

- Defining **multiple constructors** within a class with **different parameter lists**.
- Allows objects to be **initialized in various ways** depending on the constructor used.

Why Use Constructor Overloading?

- Flexibility to create objects with different initial states.
- Simplifies object creation by providing default or customizable initialization options.



Constructor Chaining

Using `this()` to call one constructor from another within the same class.

Reduces code duplication by reusing existing constructor logic.

Ensures a **single initialization point** for fields.

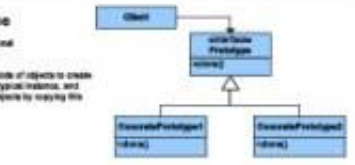
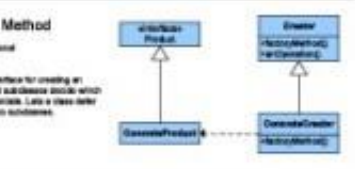
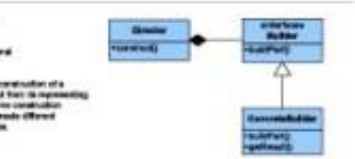
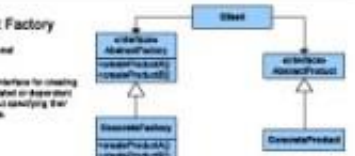
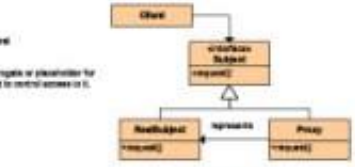
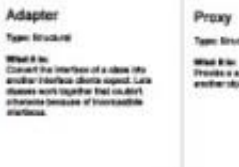
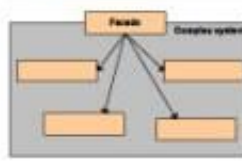
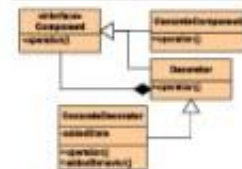
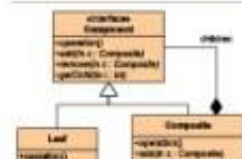
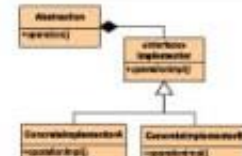
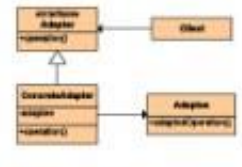
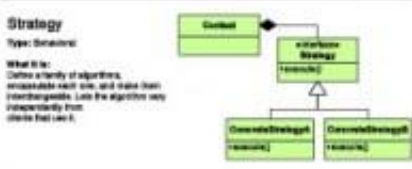
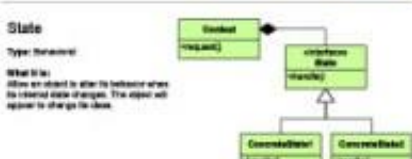
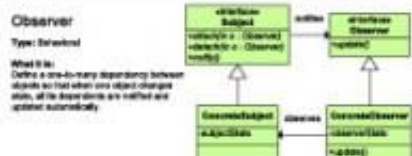
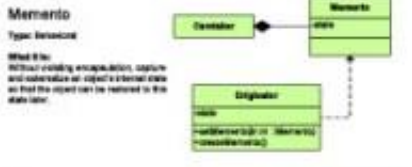
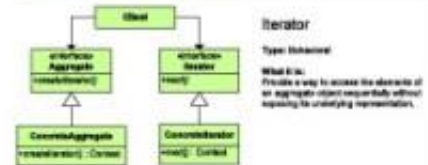
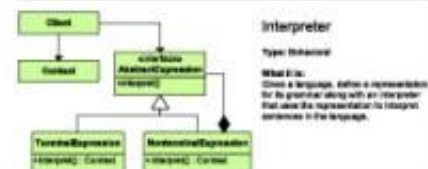
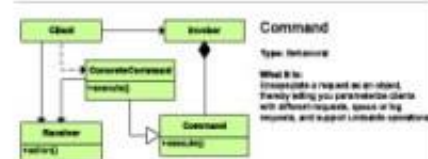
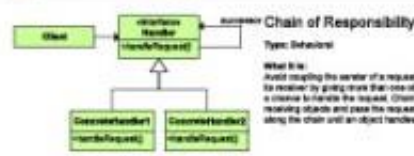
All constructors ultimately funnel into one "master" constructor.

```
public class Rectangle {  
    int width, height;  
  
    public Rectangle() {  
        this.width = 1;  
        this.height = 1; // Default  
    }  
  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
}
```

```
public Rectangle(int size) {  
    this(size, size); // Calls the two-parameter constructor  
}
```


Design Patterns

- Factory Method
- Adapter
- Bridge
- Builder
- Chain of Responsibility
- Command
- Composite
- Decorator
- Facade
- Factory Method
- Flyweight
- Interpreter
- Iterator
- Observer
- Singleton
- State
- Strategy
- Template Method
- Visitor
- Proxy
- Observer
- Singleton
- State
- Strategy
- Template Method
- Visitor



Introduction to Design Patterns

A design pattern is a **reusable solution** to a common problem in software design.
Acts as a **blueprint** for structuring code effectively.

- **Why Use Design Patterns?**

1. Promote **best practices** for writing clean, efficient, and scalable code.
2. Improve **code readability** and **maintainability**.
3. Facilitate collaboration by providing a **shared vocabulary** for solutions.

Types of Design Patterns

1. Creational Patterns:

- Focus on object creation.
- Example: Singleton, Factory.

2. Structural Patterns:

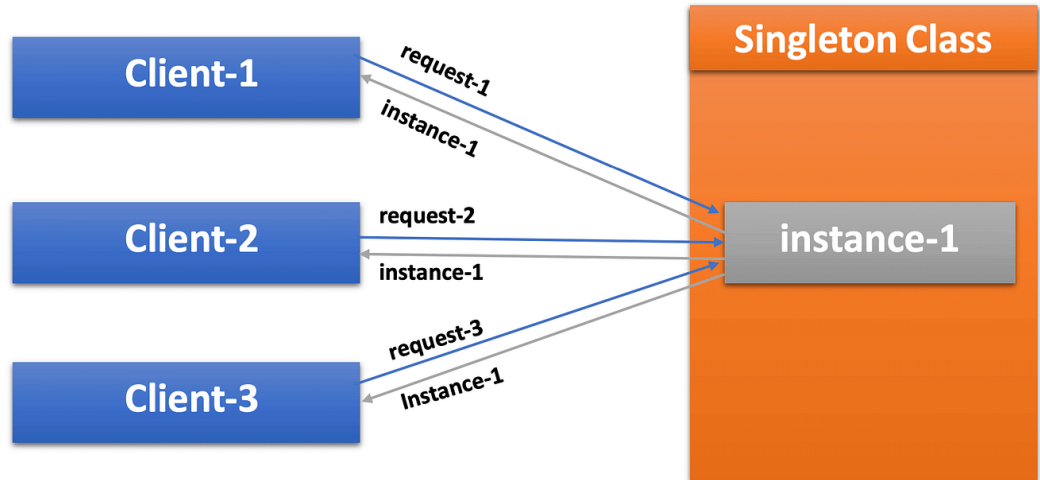
- Focus on class and object composition.
- Example: Adapter, Decorator.

3. Behavioral Patterns:

- Focus on communication between objects.
- Example: Observer, Strategy.

Singleton Pattern

- The Singleton pattern is simple and doesn't require inheritance, interfaces, or abstraction.
- It solves the problem of ensuring there's only **one instance** of a class globally.
- **Where Is It Used?**
 - Logging
 - Configuration settings
 - Managing database connections



```
public class Singleton {
    private static Singleton instance;

    // Private constructor to prevent instantiation
    private Singleton() {}

    // Method to provide access to the single instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from the Singleton!");
    }
}
```

```
public class Singleton {
    private static Singleton instance;

    // Private constructor to prevent instantiation
    private Singleton() {}

    // Method to provide access to the single instance
    public static Singleton getInstance() {
        ...
    }
}

public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        System.out.println(s1 == s2); // true (both refer to the same instance)
        s1.showMessage();
    }
}
```

Scenario:

- You are building a game with a **global configuration manager** to store game settings (e.g., difficulty, volume).

```
public class GameConfig {
    private static GameConfig config;
    private String difficulty;
    private int volume;

    private GameConfig() {
        // Default settings
        this.difficulty = "Normal";
        this.volume = 50;
    }

    public static GameConfig getInstance() {
        if (config == null) {
            config = new GameConfig();
        }
        return config;
    }

    public void setDifficulty(String difficulty) {
        this.difficulty = difficulty;
    }
}
```


Scenario:

- You are building a game with a **global configuration manager** to store game settings (e.g., difficulty, volume).

```
public class Main {  
    public static void main(String[] args) {  
        GameConfig config = GameConfig.getInstance();  
        System.out.println("Difficulty: " + config.getDifficulty());  
        System.out.println("Volume: " + config.getVolume());  
  
        config.setDifficulty("Hard");  
        config.setVolume(75);  
  
        GameConfig config2 = GameConfig.getInstance(); // Same instance  
        System.out.println("New Difficulty: " + config2.getDifficulty());  
        System.out.println("New Volume: " + config2.getVolume());  
    }  
}
```